Informe Proyecto 3: Del Pochoco SoC a un macro ASIC

Arquitectura de Computadores · Semestre 2026-2 · Universidad de los Andes · Facultad de Ingeniería y Ciencias Aplicadas

Profesor: Jorge Gómez Mir

Integrantes (Grupo 7): Lucas Fernández · Maximiliano Ferrando · Renato Rivadeneira · Matías Ureta

Repositorio: github.com/Max-Ferrando/Proyecto3_ASIC_Grupo7

Flujo y tecnología: LibreLane 3.0.16 · PDK GF180MCU (`gf180mcuD`) · biblioteca `gf180mcu_fd_sc_mcu7t5v0`

Resumen. Llevamos el Pochoco SoC del Proyecto 2 (Espino Core RV32E + periféricos + juego de reflejos) desde RTL hasta GDSII. Las dos estrategias de memoria cierran el flujo completo con T = 100 ns y FP_CORE_UTIL = 40 %: DRC 0, LVS 0, antena 0 y timing sin violaciones en todos los corners. La estrategia A (ROM combinacional + RAM de 8 palabras) ocupa un die de **0,444 mm²** (US$10.216); la B (memoria unificada de flip-flops) ocupa **2,252 mm²** (US$51.806), 5,1 veces más. Con A, el menor período cerrado es **65 ns (15,4 MHz)**; la optimización ESPINO_REG_DECODE lo baja a **60 ns (16,7 MHz)** con +3,6 % de área, y subir la utilización a 62 % reduce el die a **0,291 mm²** (US$6.699). La simulación del netlist post-síntesis entrega exactamente las mismas salidas que el RTL, ciclo a ciclo.

1. OBJETIVO Y ALCANCE

El Proyecto 3 consiste en adaptar el Pochoco SoC, que en el Proyecto 2 ejecutaba un juego de reflejos en la Go Board, para obtener un macro digital en tecnología ASIC. Usamos LibreLane y el PDK GF180MCU. El objetivo no es programar una FPGA, sino llegar desde RTL hasta un diseño físico GDSII, comparar dos opciones de memoria y justificar las decisiones con reportes medidos.

Se conserva el Espino Core RV32E y el programa del juego en Assembly. Lo que cambia es la forma de implementar la memoria, el reset y la interfaz de entrada/salida. Los pines de `asic_top` son terminales lógicos del macro; no representan pads con protección ESD ni drivers de potencia.

1.1. Cumplimiento de los requisitos del enunciado

Requisito	Implementación y resultado medido	Sección
Adaptar RTL FPGA a ASIC	`asic_top` + módulos ASIC; cada construcción asociada a su mensaje de LibreLane (archivo y línea) o explicada si no genera mensaje	2
Comparar dos memorias	A: 0,444 mm², 1.070 FF · B: 2,252 mm², 5.967 FF; ambas cierran signoff	3
Síntesis y módulos	Cinco celdas más usadas y área por módulo (banco de registros 42,6 %)	4
Timing: barrido, ruta y mejora	Cinco períodos (100/70/65/60/50 ns); menor período cerrado 65 ns; ruta crítica fetch→memoria; mejora a 60 ns	5
Floorplan y costo	Utilización 40/55/62/70 %; mínimo válido 62 % (0,291 mm², US$6.699)	6
Signoff y GDSII	DRC 0, LVS 0, antena 0 (A y B); layout anotado con banco, ALU y contador	7
RTL vs post-síntesis	Mismos estímulos en ciclos; salidas idénticas ciclo a ciclo	8
Reproducibilidad	Versiones exactas de herramientas y revisión del PDK	9

2. DISEÑO GENERAL Y ADAPTACIÓN FPGA → ASIC

2.1. Arquitectura del proyecto

Separamos el diseño en tres carpetas. `rtl/common` reúne los módulos compartidos (core, periféricos, SPI y contador). `rtl/fpga` mantiene las partes específicas de la Go Board. `rtl/asic` contiene el top-level y las alternativas de memoria y banco de registros adecuadas al flujo ASIC. Así no modificamos el core por decisiones que solo pertenecen a una tecnología, y el mismo `rtl/common` sigue funcionando en la FPGA.

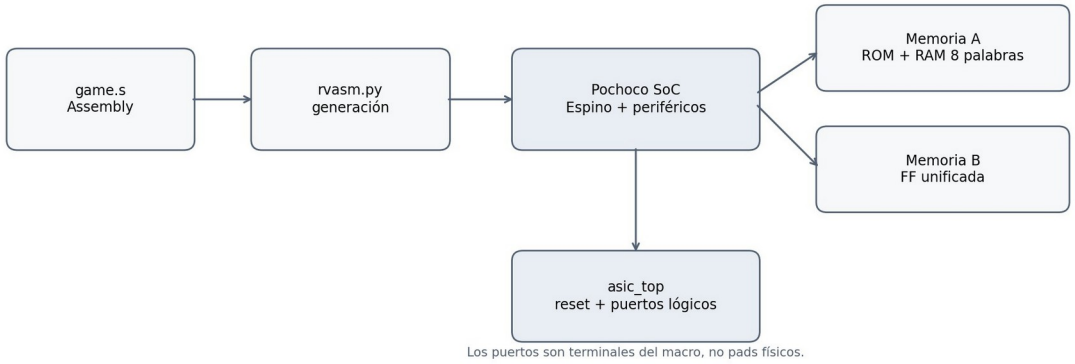


Figura 1. *Flujo lógico del proyecto: el assembler entrega el programa y se elige una de las dos memorias ASIC.*

2.2. Jerarquía y módulos principales

Módulo o archivo	Función
`asic_top.v`	Top-level ASIC; puertos lógicos de reloj, reset, LEDs, botones, displays y SPI
`pochoco_reset_sync.v`	Reset activo en bajo: activación asíncrona y liberación sincronizada con 2 FF
`pochoco_soc.v`	Interconecta el core, la memoria y los periféricos
`espino_register_file_ff.v`	Banco RV32E (x1…x15) con flip-flops y lectura registrada, igual timing que la BRAM
`pochoco_cycle_counter.v`	Contador de 32 bits accesible por software (instancia `u_cycle_cnt`)
`pochoco_mem_rom.v` / `pochoco_mem_ff.v`	Estrategias A y B de almacenamiento
`gen/game_rom.v` y `gen/game_mem.vh`	Generados automáticamente desde `game.s` por `tools/rvasm.py`

2.3. Cambios de FPGA a ASIC y evidencia en LibreLane

Para obtener la evidencia pasamos el RTL **original** del Proyecto 2, sin cambios, por LibreLane (`asic/p2_original/config.yaml`). El flujo se detiene en el primer paso por la primitiva del iCE40. Luego hicimos una segunda corrida (`config_regfile_ff.yaml`) que solo reemplaza el banco de registros, para ver qué hace la síntesis con el resto de las construcciones de FPGA.

Construcción del Proyecto 2	Mensaje de LibreLane (archivo:línea)	Por qué no sirve y cambio adoptado
`SB_RAM40_4K` en el banco de registros	Error. `01-verilator-lint/verilator-lint.log:1` y `:13`: *%Error-MODMISSING espino_register_file.v:22:3 (y 33:3): Cannot find file*	Es un bloque físico del iCE40 y no existe en GF180; el flujo se detiene. Se reemplaza por `espino_register_file_ff.v` (FF +

Construcción del Proyecto 2	Mensaje de LibreLane (archivo:línea)	Por qué no sirve y cambio adoptado
	containing module 'SB_RAM40_4K'. `05-yosys-jsonheader/yosys-jsonheader.log:474`: *ERROR: Module '\SB_RAM40_4K' referenced in module '\espino_regbank32' in cell '\u_hi' is not part of the design.*	multiplexores, un solo arreglo con dos puertos de lectura).
Power-on reset por valor inicial `reg [3:0] por_cnt = 0`	Sin error. `11-yosys-synthesis/yosys-synthesis.log:283`: *Set init value: \por_cnt = 4'0000*; línea 980: el registro se mapea a un FF sin reset.	El valor inicial lo carga el bitstream de la FPGA; un FF de silicio enciende en un valor desconocido y las celdas del PDK no tienen valor inicial. Se reemplaza por pin `rst_ni` + `pochoco_reset_sync`.
`$readmemh` + BRAM de 512 palabras	Sin error. `reports/stat.rpt`: la memoria se vuelve 16.452 `dffq_1` + 16.459 `mux2_2`; área de celdas 2.326.841,75 µm², 47,41 % secuencial.	No hay archivo que se cargue al encender: los FF no guardan el programa y la CPU ejecutaría basura. Además, solo esas celdas costarían ~US$53.500. Se reemplaza por la estrategia A o B (sección 3).
Reloj supuesto de 25 MHz en `game.s`	Sin mensaje (es software).	El macro corre a 10 MHz: con las constantes de 25 MHz la fase de 3 s duraría 7,5 s. Se ensambla con `CLK_HZ = 10.000.000` (sección 5.4).
Display activo en bajo	Sin mensaje.	Es propiedad de la Go Board, no del juego. El macro entrega lógica positiva; la inversión quedó en `rtl/fpga/game_top.v`.



Figura 2. *Corrida del RTL original del Proyecto 2: errores de Verilator y Yosys por la primitiva SB_RAM40_4K.*

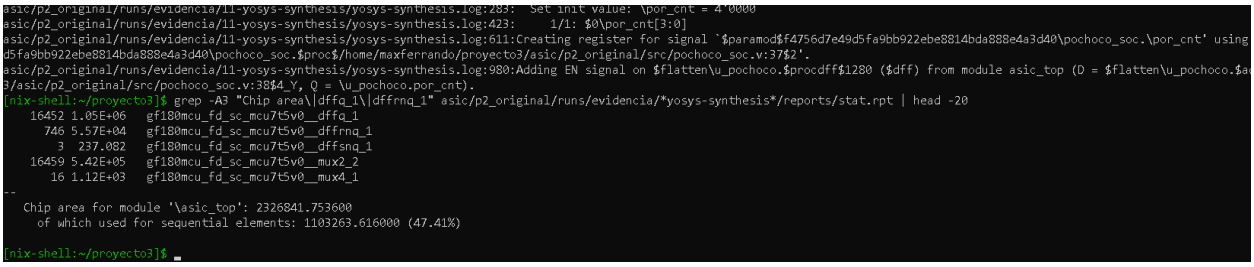


Figura 3. *Corrida con el banco reemplazado: `por_cnt` solo genera un aviso de valor inicial y la memoria con $readmemh se convierte en 16.452 FF sin contenido.*

Las construcciones que no generan error (valor inicial de `por_cnt`, `$readmemh`, reloj y polaridad del display) son igualmente un problema: el flujo las acepta, pero el chip partiría en un estado aleatorio, sin programa, con tiempos 2,5 veces más lentos o con el display invertido.

3. MEMORIA Y PROGRAMA DEL JUEGO

3.1. Uso real del programa

Medimos el uso de memoria de dos maneras. **Estática:** el assembler (`rvasm.py --report`) cuenta **153 palabras** de instrucciones (612 bytes) para el programa a 10 MHz, y las 24 instrucciones `lw`/`sw` usan como base `s0 = 0x8000_0000`, es decir, solo periféricos. **Dinámica:** la simulación RTL registra la mayor dirección pedida por el fetch, `0x258` (151 palabras con el programa de simulación), y **0 accesos** del puerto de datos a la memoria (figura 22).

Conclusión: el juego usa 153 palabras de instrucciones y **cero palabras de RAM de datos**. Reservamos 8 palabras de RAM en ambas estrategias para poder correr programas con pila o variables; esa reserva no son datos usados por el juego. La ROM tiene capacidad para 256 palabras, dentro del máximo recomendado.

3.2. Dos alternativas que se comparan

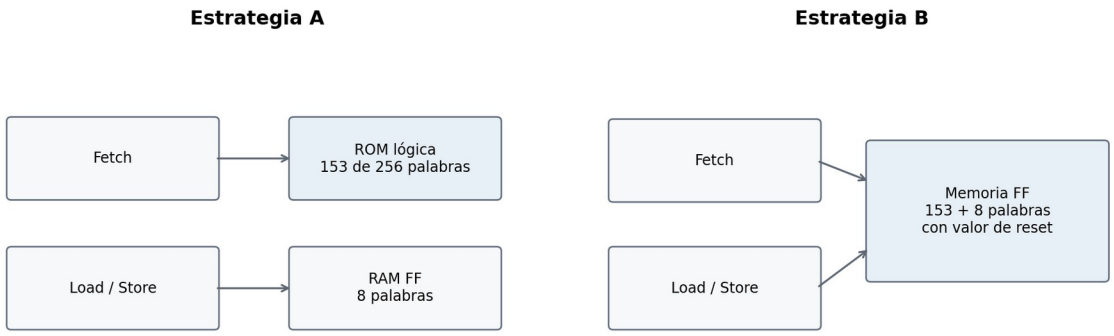


Figura 4. *Memoria A y B: la diferencia principal es almacenar las instrucciones como función lógica o como bits de flip-flops.*

Característica	A: ROM + RAM FF	B: memoria unificada FF
Programa	ROM combinacional generada por el assembler (`game_rom.v`)	FF con patrón inicial por reset (`dffrnq`/`dffsnq`)
Instrucciones	153 palabras; capacidad de la ROM 256	153 palabras dentro de la memoria unificada
Datos reservados	8 palabras FF desde `0x0000_0400`	8 palabras FF junto al programa
Lecturas	Salida de ROM registrada	Dos puertos de lectura registrados
Software `game.s`	Mismo programa	Mismo programa
Limitación	El puerto de datos no lee la ROM	Cada bit de programa requiere un FF físico

3.3. Métricas medidas de las dos memorias

Ambas corridas usan la configuración obligatoria (T = 100 ns, FP_CORE_UTIL = 40 %). Los valores salen de `final/metrics.json` de cada corrida, resumidos con `asic/scripts/metrics.py` (figuras 5 y 6).

Métrica	A (`strategy_A/runs/final`)	B (`strategy_B/runs/final_v3`)	B / A
Área del die (µm²)	444.189 (657,6 × 675,5 µm)	2.252.425	5,1×
Área de celdas (µm²)	213.569	1.347.150	6,3×
Flip-flops	1.070	5.967	5,6×
Utilización real (%)	51,58	61,66	—

Métrica	A (`strategy_A/runs/final`)	B (`strategy_B/runs/final_v3`)	B / A
Setup WS típico / lento (ns)	+65,917 / +35,267	+54,196 / +11,329	—
Hold WS típico / lento (ns)	+0,408 / +0,996	+0,324 / +0,822	—
Menor período cerrado (ns)	65 (medido, sección 5)	≈ 89 (estimado: 100 − 11,33)	—
Frecuencia máxima (MHz)	15,4	≈ 11,3 (estimada)	0,7×
Costo (US$23.000/mm²)	US$10.216	US$51.806	5,1×
Tiempo de corrida	5 min 17 s	34 min 55 s	6,6×

```
[nix-shell:~/proyecto3]$ python3 asic/scripts/metrics.py asic/strategy_A/runs/final --cols run,die_um2,cell_area_um2,uti
l_%,flip_flops,setup_ws_tt,setup_ws_ss,hold_ws_tt,hold_ws_ss,route_drc,magic_drc,lvs_errors,antenna_nets,timing_ok,signo
ff_ok
| run | die_um2 | cell_area_um2 | util_% | flip_flops | setup_ws_tt | setup_ws_ss | hold_ws_tt | hold_ws_ss | route_drc
| magic_drc | lvs_errors | antenna_nets | timing_ok | signoff_ok |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| final | 444,189 | 213569 | 51.580 | 1070 | 65.917 | 35.267 | 0.408 | 0.996 | 0 | 0 | 0 | 0 | SI | SI |
```

Figura 5. *Métricas de la corrida final de la estrategia A.*

```
[nix-shell:~/proyecto3]$ python3 asic/scripts/metrics.py asic/strategy_B/runs/final_v3 --cols run,die_um2,cell_area_um2,
util_%,flip_flops,setup_ws_tt,setup_ws_ss,hold_ws_tt,hold_ws_ss,antenna_nets,timing_ok,signoff_ok
| run | die_um2 | cell_area_um2 | util_% | flip_flops | setup_ws_tt | setup_ws_ss | hold_ws_tt | hold_ws_ss | antenna_ne
ts | timing_ok | signoff_ok |
|---|---|---|---|---|---|---|---|---|---|---|---|
| final_v3 | 2,252,425 | 1347150 | 61.660 | 5967 | 54.196 | 11.329 | 0.324 | 0.822 | 0 | SI | SI |
```

Figura 6. *Métricas de la corrida final de la estrategia B (final_v3).*

La hipótesis se confirma: B es **5,1 veces más grande** y más lenta. En A cada bit del programa es solo un término de una función combinacional que la síntesis minimiza; en B cada bit es un flip-flop con set/reset, y además hacen falta dos multiplexores de lectura de 161 entradas y redes de fanout muy alto (la red `_05870_` llega a 1.002 terminales), que alargan la ruta crítica y provocaron la violación de antena de la sección 7.2. **Elegimos la estrategia A** como diseño final; el resto del informe trabaja sobre ella.

4. SÍNTESIS Y TRAZABILIDAD JERÁRQUICA

LibreLane aplana el diseño al sintetizar. Para no perder la trazabilidad conservamos tres módulos con `SYNTH_KEEP_HIERARCHY_MODULES`: `espino_register_file`, `espino_alu` y `pochoco_cycle_counter`. Sus celdas mantienen el prefijo de la instancia hasta el DEF final (`u_soc.u_core.u_rf/…`, `u_soc.u_core.u_id.u_alu/…`, `u_soc.u_periph.u_cycle_cnt/…`), lo que permite ubicarlas en el layout (sección 7.1). Para el área de **todos** los módulos, `asic/scripts/area_by_module.py` re-sintetiza el mismo RTL con Yosys y la misma biblioteca (corner típico) manteniendo toda la jerarquía, y lee el `stat` jerárquico previo al aplanamiento.

4.1. Cinco celdas más usadas

Celda estándar	Cantidad	Función que explica su uso
`mux2_1`	994	Retención del valor en los FF del banco de registros cuando no se escriben (habilitación de escritura por registro) y selección de operandos.
`dffrnq_1`	812	Flip-flop con reset asíncrono activo en bajo: los 480 bits del banco (15 × 32), lecturas registradas, PC, registros de pipeline y contador de ciclos.
`nand2_1`	649	Árboles de los multiplexores de lectura 15:1 de 32 bits (dos puertos) y decodificación de direcciones.

Celda estándar	Cantidad	Función que explica su uso
`aoi22_1`	463	AND-OR-invertido: forma compacta de los multiplexores de lectura del banco y de la ROM.
`dffq_1`	256	FF sin reset: exactamente la RAM de datos de 8 × 32 bits. Como el juego no la usa, es área que se podría ahorrar (~25.000 µm²).

4.2. Área por bloque

Bloque	Celdas	Área (µm²)	% del total
Banco de registros (`espino_register_file`)	1.796	69.400	42,6 %
ALU (`espino_alu`)	539	8.880	5,4 %
Contador de ciclos (`pochoco_cycle_counter`)	97	3.710	2,3 %
Resto (memoria, ROM, etapas IF/ID, periféricos, SPI…)	2.897	81.010	49,7 %
Total (`asic_top`)	5.329	163.000	100 %

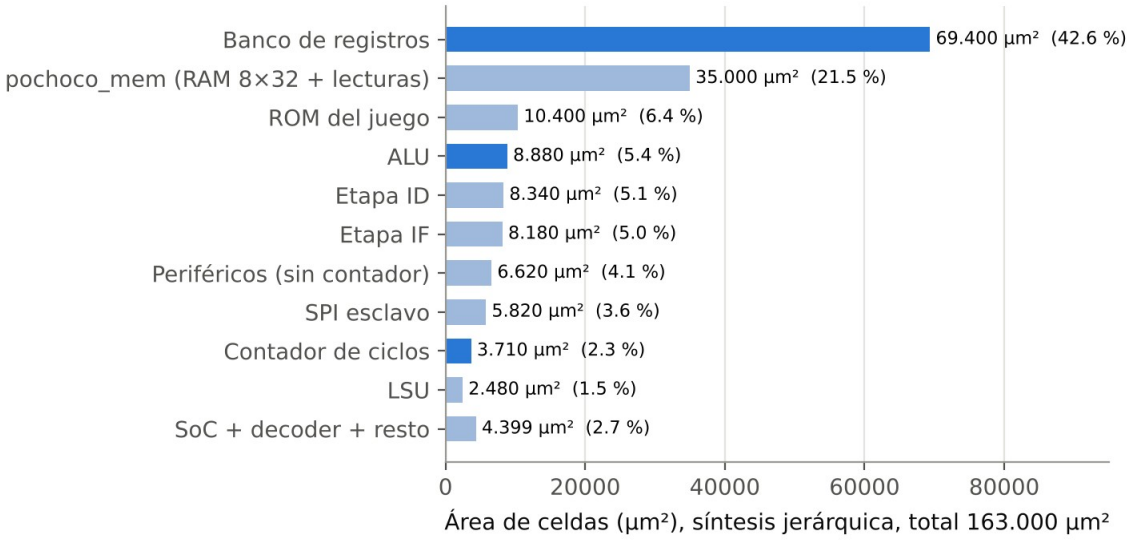


Figura 7. *Área de celdas por módulo (síntesis jerárquica, sin aplanar). En azul fuerte, los tres bloques pedidos.*

El módulo más grande es el **banco de registros (42,6 %)**, como esperábamos: 480 bits de FF más un `mux2` de retención por bit y dos multiplexores de lectura de 15 entradas × 32 bits. Le sigue la memoria (21,5 %), donde los 256 FF de la RAM pesan más que la ROM completa del juego (6,4 %). El contador de ciclos es pequeño (2,3 %): 32 FF y un sumador incremental.

El total de 163.000 µm² es menor que los 213.569 µm² de la corrida final porque el flujo físico agrega buffers de fanout, celdas de retardo, reparación de timing, celdas tap y diodos. Medido en el layout final, el banco de registros ocupa 112.109 µm² (2.191 celdas, 52 % del área de celdas).

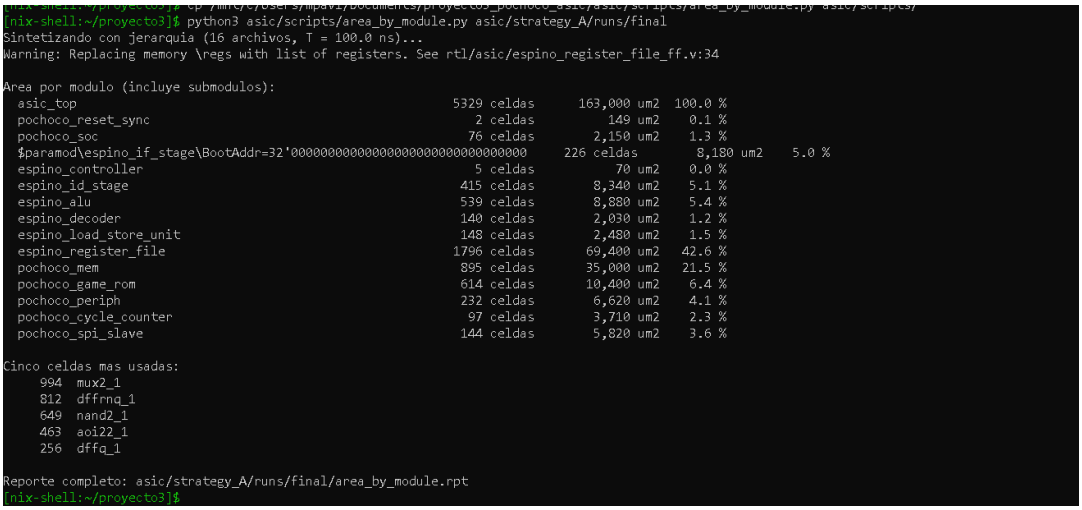


Figura 8. *Salida de `area_by_module.py` sobre la corrida final de A.*

5. TEMPORIZACIÓN, RUTA CRÍTICA Y CÁLCULOS

5.1. Barrido de período y criterio de cierre

El período objetivo es 100 ns (10 MHz). Una corrida cierra cuando WS ≥ 0 y TNS = 0 en setup y hold en **todos** los corners de STA del PDK (`metrics.py` revisa el peor de todos); en la tabla mostramos el típico (`nom_tt_025C_5v00`) y el más lento (`max_ss_125C_4v50`), que es el que limita setup. El barrido se hizo con `asic/scripts/sweep.py`.

Período (ns)	Setup típico WS / TNS	Hold típico WS / TNS	Setup lento WS / TNS	Hold lento WS / TNS	Cierra
100	+65,917 / 0	+0,408 / 0	+35,267 / 0	+0,996 / 0	Sí
70	+35,917 / 0	+0,408 / 0	+5,267 / 0	+0,996 / 0	Sí
65	+30,917 / 0	+0,408 / 0	+0,267 / 0	+0,996 / 0	Sí
60	+25,917 / 0	+0,408 / 0	−4,733 / −154,605	+0,996 / 0	No
50	+17,322 / 0	+0,408 / 0	−11,825 / −4.167	+0,997 / 0	No

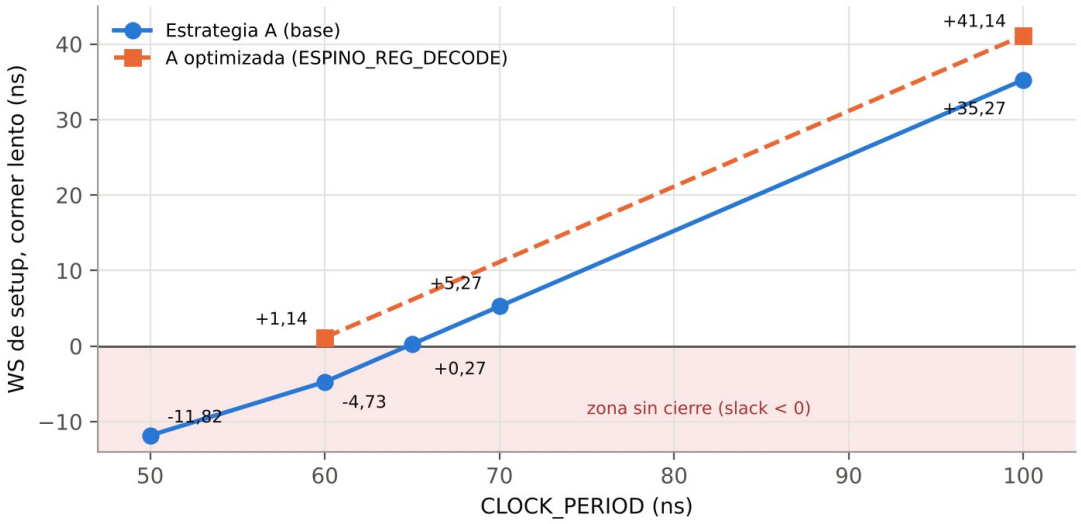


Figura 9. *Slack de setup en el corner lento según el período, para A y A optimizada.*

Menor período cerrado: 65 ns → frecuencia máxima 15,4 MHz (slack lento de solo +0,267 ns). El corner típico cierra incluso a 50 ns; lo que limita es el corner lento (proceso SS, 125 °C, 4,5 V). Un detalle importante: el área de celdas **no cambió** (213.569 µm² de 100 a 60 ns; 213.547 a 50 ns). La reparación de timing de LibreLane trabaja sobre el corner típico, donde siempre quedaba slack positivo, así que no agregó buffers ni agrandó celdas; por eso los períodos cortos fallan en el lento sin que el flujo intente arreglarlos.

```
<<< CLOCK_PERIOD_50: OK en 5.7 min

| run | CLOCK_PERIOD_ns | FP_CORE_UTIL_% | die_um2 | cell_area_um2 | util_% | setup_ws_tt | setup_tns_tt | setup_ws_ss | setup_tns_ss | hold_ws_tt | hold_tns_tt | hold_ws_ss | hold_tns_ss | route_drc | grt_overflow | antenna_nets | timing_ok | signoff_ok |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| CLOCK_PERIOD_70 | 70 | 40 | 444,189 | 213569 | 51.580 | 35.917 | 0 | 5.267 | 0 | 0.408 | 0 | 0.996 | 0 | 0 | 64 | 0 | SI | SI |
| CLOCK_PERIOD_60 | 60 | 40 | 444,189 | 213569 | 51.580 | 25.917 | 0 | -4.733 | -154.605 | 0.408 | 0 | 0.996 | 0 | 0 | 64 | 0 | NO | SI |
| CLOCK_PERIOD_50 | 50 | 40 | 444,189 | 213547 | 51.580 | 17.322 | 0 | -11.825 | -4,167 | 0.408 | 0 | 0.997 | 0 | 0 | 69 | 0 | NO | SI |

Tabla completa: /home/maxferrando/proyecto3/asic/strategy_A/sweep_CLOCK_PERIOD.csv
  CLOCK_PERIOD_70: codigo 0, 5.7 min
  CLOCK_PERIOD_60: codigo 0, 5.6 min
  CLOCK_PERIOD_50: codigo 0, 5.7 min
[nix-shell:~/proyecto3]$
```

Figura 10. *Barrido de CLOCK_PERIOD 70/60/50 ns (estrategia A).*

```
[nix-shell:~/proyecto3]$ python3 asic/scripts/metrics.py asic/strategy_A/runs/CLOCK_PERIOD_65 --cols run,setup_ws_tt,setup_tns_tt,setup_ws_ss,setup_tns_ss,hold_ws_tt,hold_ws_ss,timing_ok
| run | setup_ws_tt | setup_tns_tt | setup_ws_ss | setup_tns_ss | hold_ws_tt | hold_ws_ss | timing_ok |
|---|---|---|---|---|---|---|---|
| CLOCK_PERIOD_65 | 30.917 | 0 | 0.267 | 0 | 0.408 | 0.996 | SI |
[nix-shell:~/proyecto3]$
```

Figura 11. *Corrida a 65 ns: el menor período que cierra.*

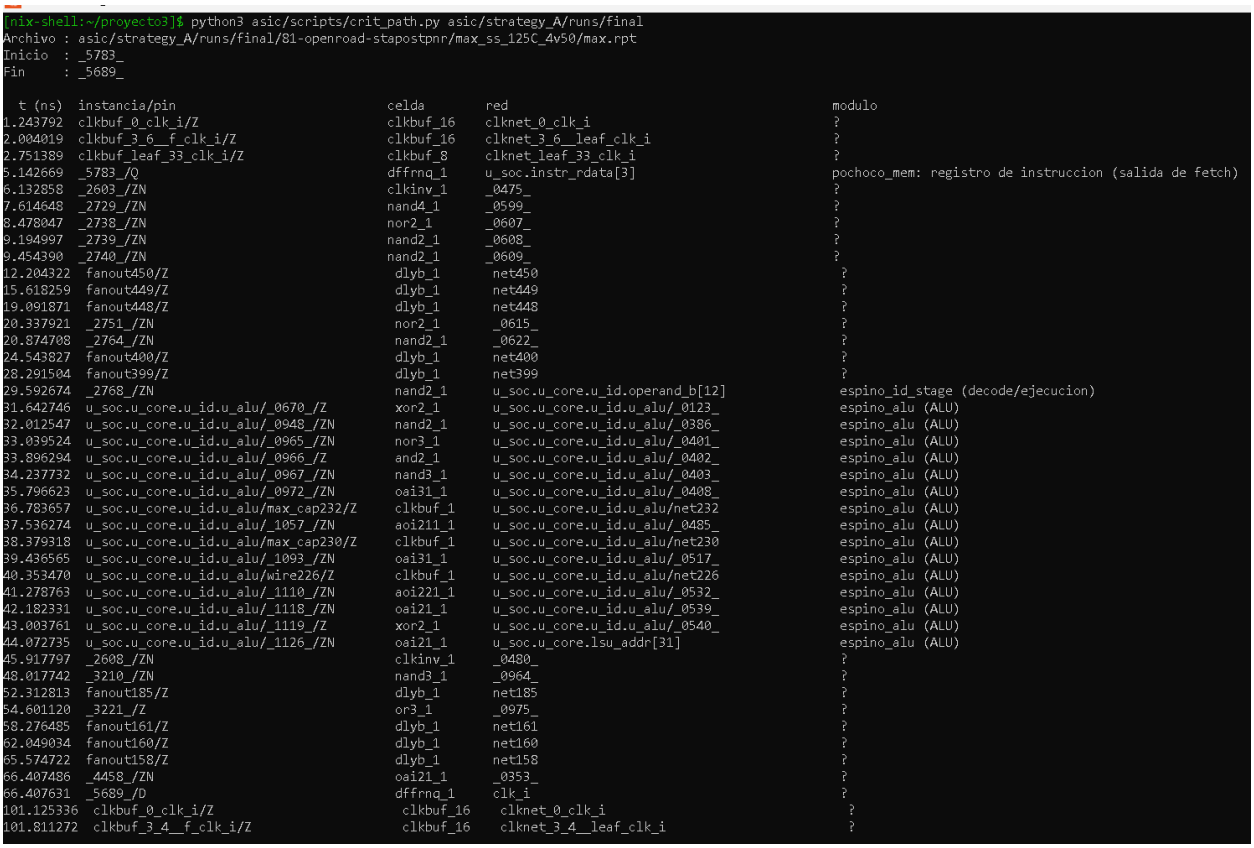
El flujo también reporta avisos de *max slew* y *max capacitance* en los corners, debidos a redes de fanout alto (por ejemplo `clk_i`, con 1.071 terminales antes del árbol de reloj). No afectan el criterio de cierre (setup/hold y signoff limpios), pero son la siguiente mejora natural.

5.2. Ruta crítica

Ruta extraída con `asic/scripts/crit_path.py` del reporte `81-openroad-stapostpnr/max_ss_125C_4v50/max.rpt` de la corrida final de A (T = 100 ns).

Inicio	Fin	Etapas de Espino	Llegada / slack
`_5783_` (`dffrnq_1`): `u_soc.instr_rdata[3]`, registro de instrucción (salida del fetch)	`_5689_/D` (`dffrnq_1`): `u_soc.mem_b_rdata[22]`, registro de lectura de la RAM de datos	Fetch → decode → ejecución (la ALU calcula la dirección de un load/store) → memoria	66,41 ns / +35,27 ns

Tramo	Tiempo (ns)	Qué ocurre
Árbol de reloj + clk→Q	0 → 5,14	Buffers de reloj y salida del registro de instrucción
Decodificación	5,14 → 9,45	Bits de la instrucción a señales de control (nand4, nor2, nand2)
Buffers de fanout	9,45 → 29,59	3 + 2 celdas `dlyb_1` y lógica hasta `operand_b[12]` (multiplexor de operandos)
ALU (`u_alu`)	29,59 → 44,07	Sumador: 15 celdas hasta `lsu_addr[31]`
Decodificación de dirección	44,07 → 66,41	Selección RAM/periféricos y 4 `dlyb_1` hasta el registro de lectura



```
[nix-shell:~/proyecto3]$ python3 asic/scripts/crit_path.py asic/strategy_A/runs/final
Archivo : asic/strategy_A/runs/final/81-openroad-stapostpnr/max_ss_125C_4v50/max.rpt
Inicio  : _5783_
Fin     : _5689_

  t (ns)  instancia/pin                          celda       red                              modulo
1.243792  clkbuf_0_clk_i/Z                       clkbuf_16   clknet_0_clk_i                   ?
2.004019  clkbuf_3_6__f_clk_i/Z                  clkbuf_16   clknet_3_6__leaf_clk_i           ?
2.751389  clkbuf_leaf_33_clk_i/Z                 clkbuf_8    clknet_leaf_33_clk_i             ?
5.142669  _5783_/Q                              dffrnq_1    u_soc.instr_rdata[3]             pochoco_mem: registro de instruccion (salida de fetch)
6.132858  _2603_/ZN                             clkinv_1    _0475_                           ?
7.614648  _2729_/ZN                             nand4_1     _0599_                           ?
8.478047  _2738_/ZN                             nor2_1      _0607_                           ?
9.194997  _2739_/ZN                             nand2_1     _0608_                           ?
9.454390  _2740_/ZN                             nand2_1     _0609_                           ?
12.204322  fanout450/Z                          dlyb_1       net450                           ?
15.618259  fanout449/Z                          dlyb_1       net449                           ?
19.091871  fanout448/Z                          dlyb_1       net448                           ?
20.337921  _2751_/ZN                            nor2_1       _0615_                           ?
20.874708  _2764_/ZN                            nand2_1      _0622_                           ?
24.543827  fanout400/Z                          dlyb_1       net400                           ?
28.291504  fanout399/Z                          dlyb_1       net399                           ?
29.592674  _2768_/ZN                            nand2_1      u_soc.u_core.u_id.operand_b[12]   espino_id_stage (decode/ejecucion)
31.642746  u_soc.u_core.u_id.u_alu/_0670_/Z     xor2_1       u_soc.u_core.u_id.u_alu/_0123_   espino_alu (ALU)
32.012547  u_soc.u_core.u_id.u_alu/_0948_/ZN    nand2_1      u_soc.u_core.u_id.u_alu/_0386_   espino_alu (ALU)
33.039524  u_soc.u_core.u_id.u_alu/_0965_/ZN    nor3_1       u_soc.u_core.u_id.u_alu/_0401_   espino_alu (ALU)
33.896294  u_soc.u_core.u_id.u_alu/_0966_/Z     and2_1       u_soc.u_core.u_id.u_alu/_0402_   espino_alu (ALU)
34.237732  u_soc.u_core.u_id.u_alu/_0967_/ZN    nand3_1      u_soc.u_core.u_id.u_alu/_0403_   espino_alu (ALU)
35.796623  u_soc.u_core.u_id.u_alu/_0972_/ZN    oai31_1      u_soc.u_core.u_id.u_alu/_0408_   espino_alu (ALU)
36.783657  u_soc.u_core.u_id.u_alu/max_cap232/Z clkbuf_1     u_soc.u_core.u_id.u_alu/net232   espino_alu (ALU)
37.536274  u_soc.u_core.u_id.u_alu/_1057_/ZN    aoi211_1     u_soc.u_core.u_id.u_alu/_0485_   espino_alu (ALU)
38.379318  u_soc.u_core.u_id.u_alu/max_cap230/Z clkbuf_1     u_soc.u_core.u_id.u_alu/net230   espino_alu (ALU)
39.436565  u_soc.u_core.u_id.u_alu/_1093_/ZN    oai31_1      u_soc.u_core.u_id.u_alu/_0517_   espino_alu (ALU)
40.353470  u_soc.u_core.u_id.u_alu/wire226/Z    clkbuf_1     u_soc.u_core.u_id.u_alu/net226   espino_alu (ALU)
41.278763  u_soc.u_core.u_id.u_alu/_1110_/ZN    aoi221_1     u_soc.u_core.u_id.u_alu/_0532_   espino_alu (ALU)
42.182331  u_soc.u_core.u_id.u_alu/_1118_/ZN    oai21_1      u_soc.u_core.u_id.u_alu/_0539_   espino_alu (ALU)
43.003761  u_soc.u_core.u_id.u_alu/_1119_/Z     xor2_1       u_soc.u_core.u_id.u_alu/_0540_   espino_alu (ALU)
44.072735  u_soc.u_core.u_id.u_alu/_1126_/ZN    oai21_1      u_soc.u_core.lsu_addr[31]        espino_alu (ALU)
45.917797  _2608_/ZN                            clkinv_1     _0480_                           ?
48.017742  _3210_/ZN                            nand3_1      _0964_                           ?
52.312813  fanout185/Z                          dlyb_1       net185                           ?
54.601120  _3221_/Z                             or3_1        _0975_                           ?
58.276485  fanout161/Z                          dlyb_1       net161                           ?
62.049034  fanout160/Z                          dlyb_1       net160                           ?
65.574722  fanout158/Z                          dlyb_1       net158                           ?
66.407486  _4458_/ZN                            oai21_1      _0353_                           ?
66.407631  _5689_/D                             dffrnq_1     clk_i                            ?
101.125336  clkbuf_0_clk_i/Z                     clkbuf_16    clknet_0_clk_i                    ?
101.811272  clkbuf_3_4__f_clk_i/Z                clkbuf_16    clknet_3_4__leaf_clk_i            ?
```

Figura 12. *Ruta crítica del corner lento con la instancia, celda, red y módulo de cada etapa.*

La ruta recorre todo el procesador en un ciclo: de la instrucción a la dirección de memoria. Lo más llamativo es que **9 celdas** `dlyb_1` **suman cerca de 30 ns** (45 % de la ruta): el reparador las usó como buffers de fanout, pero son celdas de retardo. Prohibir esas celdas en la reparación de fanout o registrar la dirección sería una mejora adicional.

5.3. Optimización evaluada

Implementamos la variante `ESPINO_REG_DECODE` (`strategy_A_opt`, `VERILOG_DEFINES`). Como el banco de registros tiene lectura registrada, cada instrucción pasa un ciclo de espera en la etapa ID. En ese ciclo calculamos la decodificación y registramos sus salidas en el mismo flanco que la lectura del banco: el decoder sale de la ruta instrucción → decoder → multiplexor de operandos → ALU **sin agregar ciclos**. El juego mide exactamente los mismos ciclos (14.349 y 8.778).

Comparación (T = 100 ns, 40 %)	Base A	A optimizada	Cambio
Área de celdas (µm²)	213.569	221.256	+3,6 %
Área del die (µm²)	444.189	458.601	+3,2 %
Setup WS típico (ns)	+65,917	+69,059	+3,14
Setup WS lento (ns)	+35,267	+41,141	+5,87
Hold WS lento (ns)	+0,996	+0,984	≈ igual
Período mínimo cerrado	65 ns (60 ns falla: −4,733)	60 ns (cierra: +1,141)	−5 ns
Frecuencia máxima	15,4 MHz	16,7 MHz	+8 %

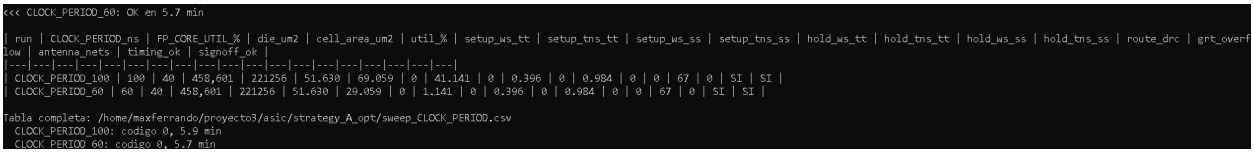


Figura 13. *Barrido de la variante optimizada: a 60 ns cierra con +1,141 ns en el corner lento.*

La mejora es real pero acotada (+8 % de frecuencia por +3,6 % de área), porque la ruta crítica sigue dominada por el sumador y la decodificación de direcciones de load/store, que esta optimización no toca.

5.4. Constantes del juego al pasar de 25 a 10 MHz

El juego usa el contador de ciclos para medir tiempos. En la Go Board cada ciclo dura 40 ns (25 MHz); con T = 100 ns la frecuencia es 10 MHz. Para mantener la fase de 3 s y el display en décimas cambiamos solo `CLK_HZ` y el assembler recalcula el resto (`make asic-gen`).

Constante	Expresión	FPGA 25 MHz	ASIC 10 MHz	Uso
CLK_HZ	—	25.000.000	10.000.000	Ciclos por segundo
TENTH	CLK_HZ / 10	2.500.000	1.000.000	0,1 s
HALF_TENTH	TENTH / 2	1.250.000	500.000	Redondeo
ALL_ON_TIME	30 · TENTH	75.000.000	30.000.000	Fase de 3 s
DEBOUNCE	TENTH / 5	500.000	200.000	20 ms
MAX_CYCLES	99 · TENTH	247.500.000	99.000.000	Límite de 9,9 s
Divisor promedio	10 · TENTH	25.000.000	10.000.000	10 rondas en décimas

Comprobación: 3 s × 10.000.000 ciclos/s = 30.000.000 ciclos y una décima son 1.000.000 ciclos. La suma máxima de 10 rondas (990.000.000) cabe en 32 bits (4.294.967.295); el contador da la vuelta cada $2^{32} / 10^7$ = 429,5 s.

6. FLOORPLAN, PLACEMENT Y COSTO

La configuración base usa FP_CORE_UTIL = 40 %. Exploramos 55, 62 y 70 % con `sweep.py` (estrategia A, T = 100 ns). No basta con el die geométricamente más pequeño: también deben cumplirse timing, DRC/LVS/antena y el ruteo tiene que converger.

Utilización objetivo	Die (µm²)	Utilización real	Overflow ruteo global	Ruteo detallado	Setup WS lento	Cumple	Costo
40 % (base)	444.189	51,58 %	64	0 DRC	+35,267	Sí	US$10.216
55 %	326.784	69,74 %	39	0 DRC	+40,566	Sí	US$7.516
62 %	291.282	78,85 %	98	0 DRC	+40,460	Sí	US$6.699
70 %	—	—	—	66.177 → 3.867 violaciones, no converge	—	No	—

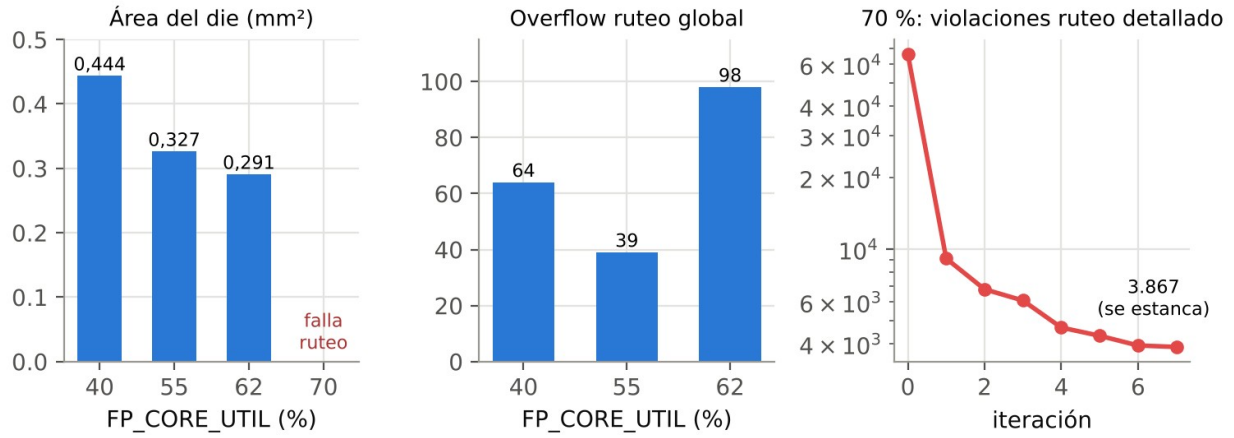


Figura 14. *Barrido de utilización: área del die, overflow del ruteo global y violaciones del ruteo detallado a 70 %. (El overflow de 40 % viene de las corridas del barrido de reloj, que usan el mismo floorplan.)*

La menor área válida es con 62 %: 0,291 mm² (34 % menos que la base). A 70 % el ruteo detallado partió con 66.177 violaciones y tras varias iteraciones quedó estancado cerca de 3.867: no hay suficientes pistas de metal para conectar todo, así que la corrida no cierra; por eso no tenía sentido correr 85 %. El timing incluso mejora al compactar (cables más cortos), y el overflow del ruteo global no es monótono porque es una estimación previa que el ruteo detallado resuelve (DRC 0 hasta 62 %).

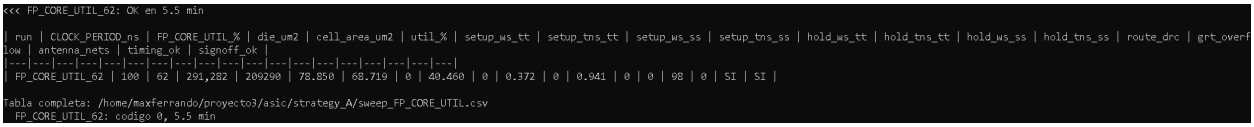


Figura 15. *Corrida a 62 %: cierra timing y signoff.*

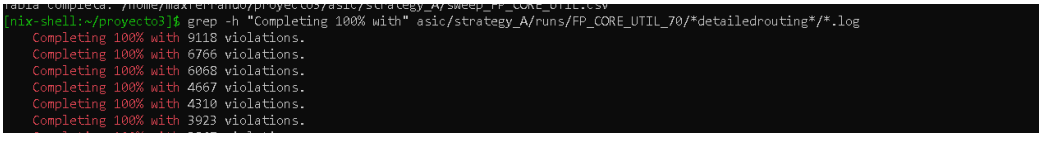


Figura 16. *Corrida a 70 %: violaciones por iteración del ruteo detallado.*

6.1. Terminales del macro

`asic/pin_order.cfg` agrupa los terminales por función, para facilitar la conexión con otros bloques: reloj y reset solos en el lado oeste, botones al norte, LEDs y displays al sur, SPI al este. Estos terminales no reemplazan los pads externos que necesitaría un chip encapsulado. El archivo no admite comentarios: un # se interpreta como lado (por eso la explicación quedó en `asic/pin_order.md`).

Lado	Contenido de pin_order.cfg (una entrada por línea)	Motivo
Oeste (#W)	`$4`, `clk_i`, `rst_ni`	Reloj y reset aislados del resto
Norte (#N)	`@min_distance=8`, `btn_i.*`	Entradas de botones
Sur (#S)	`@min_distance=4`, `led_o.*`, `seg1_o.*`, `seg2_o.*`	Salidas visibles (LEDs y displays)
Este (#E)	`@min_distance=8`, `spi_*`	Interfaz SPI

6.2. Costo equivalente didáctico

La pauta propone US$23.000 por mm² con relación lineal: costo = área del die (mm²) × 23.000. No incluye rendimiento, pads, encapsulado ni costos fijos.

Diseño	Área del die (mm²)	Costo equivalente
A, 40 % (configuración obligatoria)	0,444	US$10.216
A, 62 % (mínimo válido)	0,291	US$6.699
A optimizada, 40 %	0,459	US$10.548
B, 40 %	2,252	US$51.806
RTL del Proyecto 2 sin adaptar (solo área de celdas)	≥ 2,327	≥ US$53.500

La decisión con mayor impacto en el costo es la **estrategia de memoria** (B multiplica el costo por 5,1); la segunda es la **utilización** (−34 % de 40 a 62 %).

7. SIGNOFF Y LAYOUT

Comprobación	Estrategia A (`runs/final`)	Estrategia B (`runs/final_v3`)	Evidencia
DRC (Magic)	0 errores	0 errores	Paso *Report Manufacturability*: DRC Passed
DRC de ruteo	0	0	`route__drc_errors` en `metrics.json`
LVS (Netgen)	0 errores	0 errores	LVS Passed
Antena	0 redes	0 redes (v1 y v2: 1 red, ver 7.2)	Antenna Passed
Timing (todos los corners)	Cierra	Cierra	`timing_ok = SI`
GDSII	`final/gds/asic_top.gds`	`final/gds/asic_top.gds`	80/80 etapas, 5:17 (A) y 34:55 (B)

LibreLane omite el DRC de KLayout para `gf180mcuD` porque el PDK no define `KLAYOUT_DRC_RUNSET` (aviso *klayout__drc_error__count not reported*); el DRC de signoff es el de Magic, que da 0. También aparece el aviso GRT-0281 (`clk_i` con fanout 1.071 en el placement global), que es normal antes de construir el árbol de reloj.

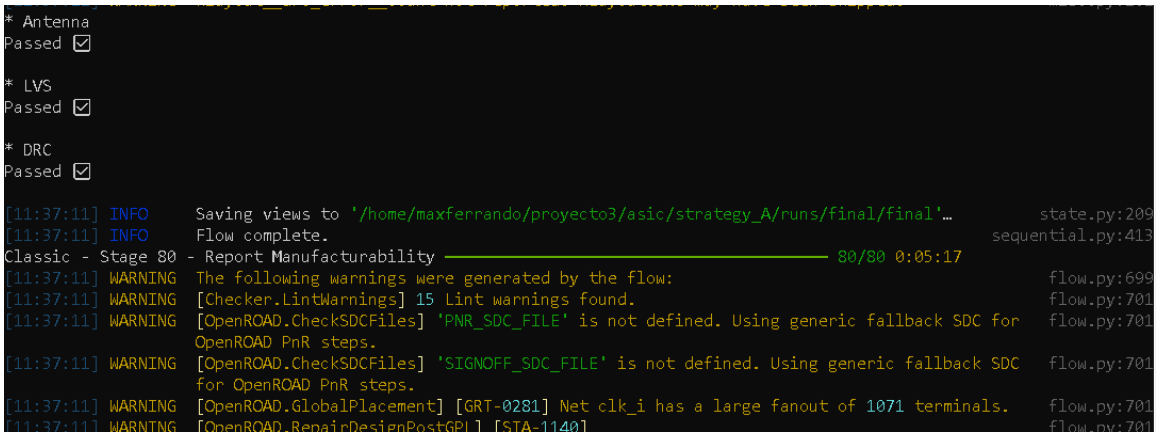


Figura 17. *Estrategia A: Antenna, LVS y DRC aprobados; flujo completo en 5 min 17 s.*

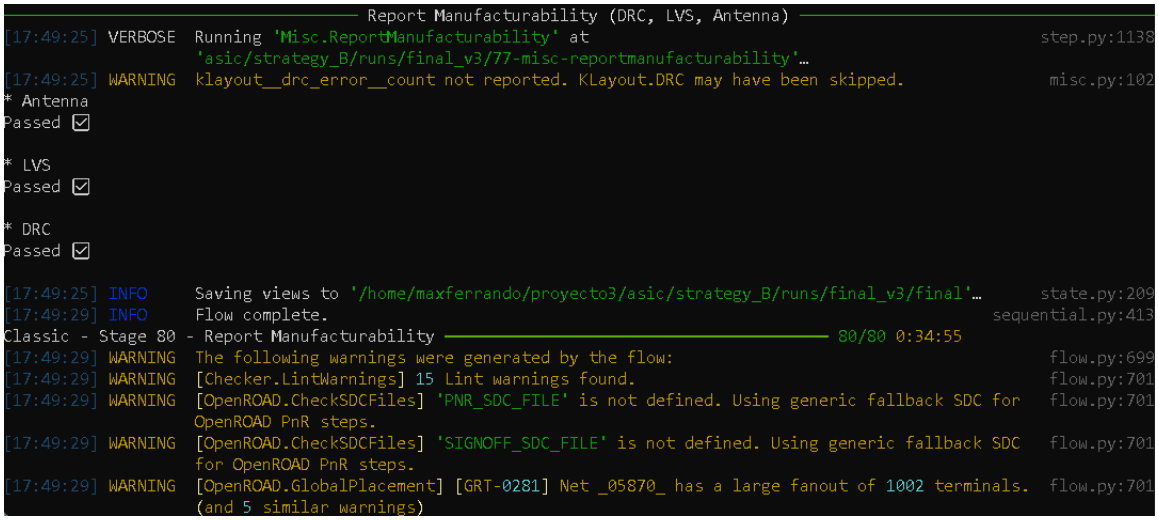


Figura 18. *Estrategia B (final_v3): Antenna, LVS y DRC aprobados; 34 min 55 s.*

7.1. Identificación visual de bloques

Como los tres módulos conservan su jerarquía (sección 4), `asic/scripts/highlight_layout.py` lee del DEF final la posición de cada celda cuyo nombre empieza con el prefijo de la instancia, la dibuja en una capa de marca de color sobre el GDS final y agrega el rectángulo que las encierra. El resultado se abrió en KLayout.

Bloque (prefijo de instancia)	Celdas	Área (µm²)	Rectángulo (µm)
Banco de registros (`u_soc.u_core.u_rf/`)	2.191	112.109	(8, 39) – (345, 608)
ALU (`u_soc.u_core.u_id.u_alu/`)	615	15.843	(254, 212) – (428, 420)
Contador de ciclos (`u_soc.u_periph.u_cycle_cnt/`)	117	5.390	(494, 278) – (635, 448)

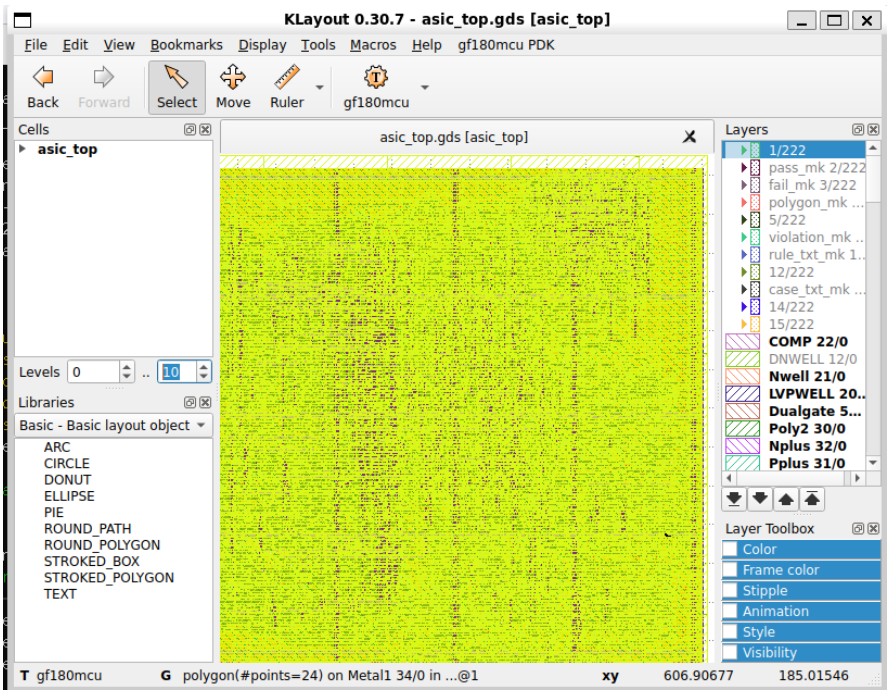


Figura 19. *GDS final de la estrategia A abierto en KLayout 0.30.7.*

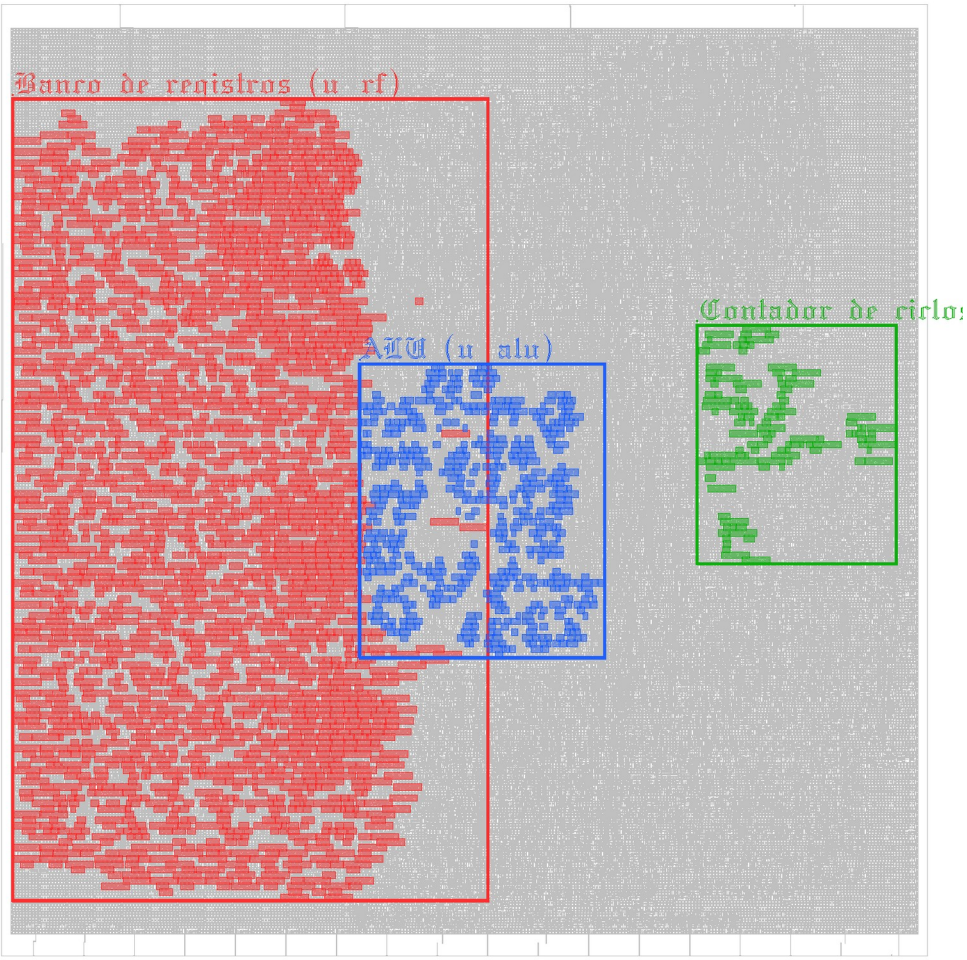


Figura 20. *Layout final anotado: banco de registros (rojo), ALU (azul) y contador de ciclos (verde).*

El banco de registros ocupa casi toda la mitad izquierda del die, coherente con ser el bloque más grande. El placer dejó la ALU pegada a él, porque sus operandos salen directamente del banco, y el contador de ciclos a la derecha, cerca de los periféricos y de los terminales SPI del lado este.

7.2. Violación de antena en la estrategia B

Al grabar el metal, una pista larga conectada solo a la compuerta de un transistor acumula carga y puede dañar el óxido: es el **efecto antena**. En la estrategia B, la red `_05410_` (pin `fanout4273/I`, celda `dlyb_1`) superaba la razón de área permitida en Metal2.

Corrida	Cambio en `config.yaml`	Resultado
final (v1)	Configuración base	1 red con violación (27 min 28 s)
final_v2	`GRT_ANTENNA_REPAIR_ITERS: 10`, `GRT_ANTENNA_REPAIR_MARGIN: 20`, `DIODE_ON_PORTS: in`	La misma red sigue fallando
final_v3	Además `DRT_ANTENNA_REPAIR_ITERS: 10`, `DRT_ANTENNA_REPAIR_MARGIN: 20`, `RUN_HEURISTIC_DIODE_INSERTION: true`	0 violaciones (34 min 55 s)

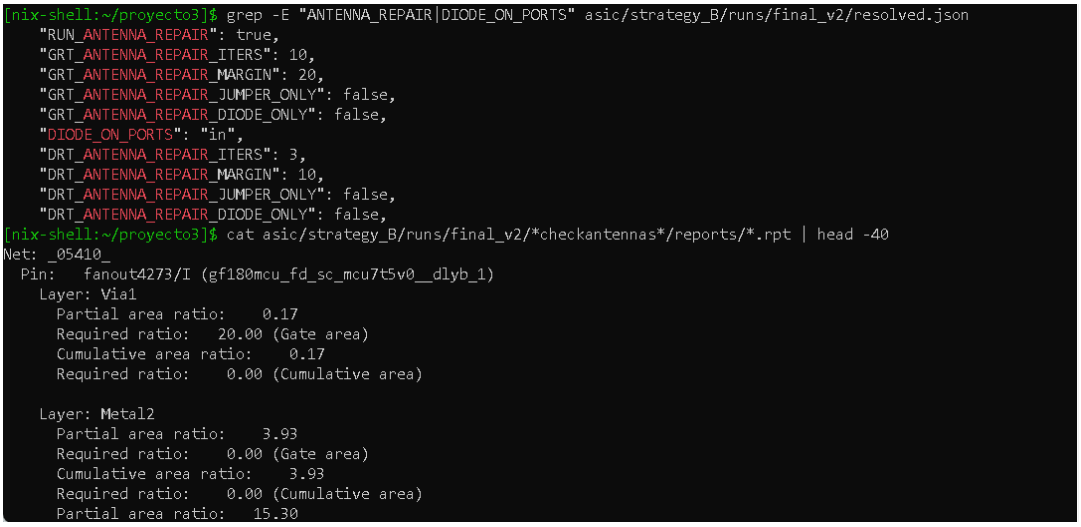


Figura 21. *Opciones de reparación de antena en final_v2 y reporte de la red `_05410_`.*

La reparación en el ruteo global no bastó; la solución fue repetirla en el ruteo detallado e insertar diodos por heurística, que descargan la pista. En v3 el STA terminó con SIGKILL por falta de memoria (WSL tenía 3,6 GB); subimos a 5 GB + 8 GB de swap en `.wslconfig` y corrimos con `--jobs 2`. La estrategia A nunca tuvo violaciones de antena.

8. SIMULACIÓN FUNCIONAL POST-SÍNTESIS

El testbench `sim/asic/tb_asic.v` aplica el mismo archivo de estímulos (`sim/asic/stim.txt`, ciclo absoluto de cada pulsación) al RTL y al netlist sintetizado con los modelos Verilog de las celdas GF180 (`asic/scripts/sim.sh`). Para que la simulación sea rápida se ensambla una versión del juego con `CLK_HZ = 20.000` (1 décima = 2.000 ciclos, 3 s = 60.000 ciclos), idéntica en ambos casos. El testbench guarda la traza de LEDs y displays en cada ciclo y compara las dos.

Caso	RTL	Netlist post-síntesis (A)	Archivo de prueba
Pulsación 1, correcta	14.349 ciclos → 07	14.349 ciclos → 07	`stim.txt`, pulsación 1
Pulsación 2, incorrecta	EE; 6.006 ciclos	EE; 6.006 ciclos	`stim.txt`, pulsación 2
Ronda reiniciada	s1 se mantiene en 1 (no pierde la correcta)	Igual	`stim.txt`, pulsación 2
Pulsación 3, correcta	8.778 ciclos → 04	8.778 ciclos → 04	`stim.txt`, pulsación 3
Final	rondas_correctas = 2, display 04	Igual	—
Trazas de LEDs y displays	—	Idénticas ciclo a ciclo	`sim/asic/gen/`



Figura 22. *Simulación RTL y del netlist de la estrategia A con los mismos estímulos: resultados idénticos.*

La pulsación incorrecta muestra EE y reinicia la ronda actual sin perder las rondas correctas, igual que en la FPGA. El RTL de la estrategia B entrega los mismos resultados.

9. REPRODUCIBILIDAD Y ENTORNO

Parámetro / herramienta	Valor usado
PDK	`gf180mcuD`, revisión `54435919abffb937387ec956209f9cf5fd2dfbee` (2025.12.26), instalado por ciel
Biblioteca estándar	`gf180mcu_fd_sc_mcu7t5v0`
CLOCK_PERIOD / FP_CORE_UTIL	100 ns / 40 %
Top-level	`asic_top`
LibreLane	3.0.16 (nix-shell en WSL2 Ubuntu)
Yosys	0.62 (git 7326bb7d)
OpenROAD	`dcf36133a369abc8f3c5e5738cd4d82e4903c0e0`
Magic / Netgen	8.3.623 / 1.5.316
KLayout / Verilator	0.30.7 / 5.044

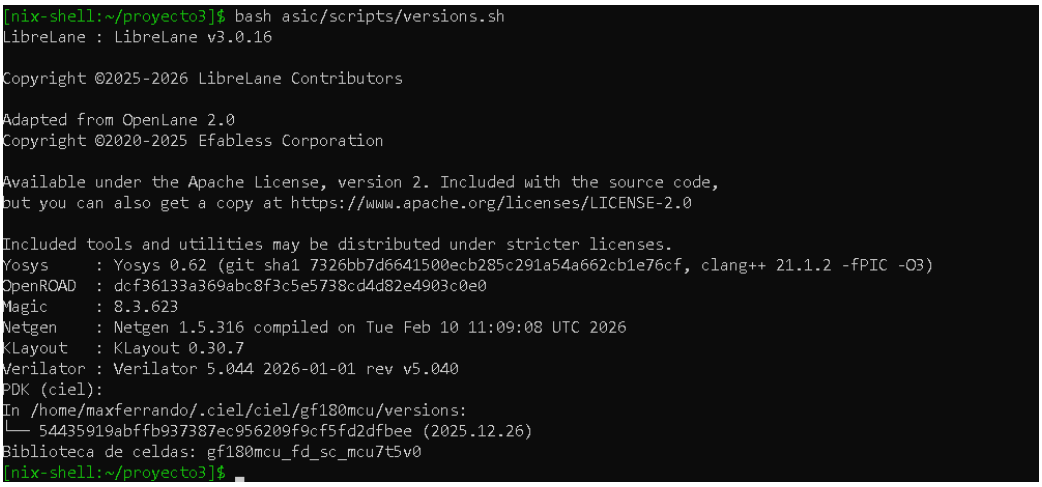


Figura 23. *Salida de `asic/scripts/versions.sh` en el entorno de las corridas finales.*

Comandos para reproducir todo desde la raíz del repositorio (dentro de `nix-shell`). El `Makefile` ya pasa `--pdk gf180mcuD`, necesario para que LibreLane descargue el PDK correcto:

```
make asic-gen                     # ROM / memoria desde game.s a 10 MHz
make asic-A  asic-B  asic-A-opt   # corridas completas RTL -> GDSII
make asic-sim  asic-synth-sim     # simulacion RTL y post-sintesis
python3 asic/scripts/sweep.py asic/strategy_A/config.yaml CLOCK_PERIOD 70 65 60 50
python3 asic/scripts/sweep.py asic/strategy_A/config.yaml FP_CORE_UTIL 55 62 70
python3 asic/scripts/metrics.py asic/strategy_A/runs/*
python3 asic/scripts/crit_path.py asic/strategy_A/runs/final
python3 asic/scripts/area_by_module.py asic/strategy_A/runs/final
python3 asic/scripts/highlight_layout.py asic/strategy_A/runs/final
bash asic/scripts/versions.sh
```

10. CONCLUSIONES

Separar el código común del específico de FPGA permitió llevar el Pochoco SoC a ASIC sin rehacer el procesador. Las diferencias de fondo fueron la memoria física, el reset y los terminales: en un ASIC no se puede depender de la inicialización que hace el bitstream. LibreLane solo detiene el flujo con la primitiva `SB_RAM40_4K`; el valor inicial de `por_cnt` y `$readmemh` pasan sin error, pero producirían un chip que parte en un estado aleatorio y sin programa.

La estrategia de memoria es la decisión de arquitectura que más pesa: guardar el programa como función lógica (A) en vez de flip-flops (B) reduce el die 5,1 veces (0,444 frente a 2,252 mm²), sube la frecuencia máxima de ~11,3 a 15,4 MHz y evita los problemas de fanout y antena de B. Dentro de A, el banco de registros es el 42,6 % del área.

El diseño final cumple con holgura los 100 ns pedidos (+35,3 ns en el corner lento) y cierra hasta 65 ns. Registrar la decodificación en el ciclo de espera del banco lleva el mínimo a 60 ns por +3,6 % de área, y subir la utilización a 62 % deja el macro en 0,291 mm² (US$6.699) con signoff limpio; 70 % ya no rutea. Todos los resultados vienen de corridas propias con DRC, LVS y antena en cero, y la simulación del netlist coincide ciclo a ciclo con el RTL.

Como trabajo futuro quedan tres mejoras concretas que vimos en los reportes: evitar que el reparador use celdas `dlyb_1` como buffers (≈30 ns de la ruta crítica), eliminar la RAM de 8 palabras que el juego no usa (*25.000 µm²) y resolver los avisos de* max slew/max cap~ de las redes de fanout alto.

ANEXO A. ARCHIVOS QUE RESPALDAN EL INFORME

Ruta	Uso
`asic/strategy_A/config.yaml`	ROM + RAM, 100 ns y 40 %, jerarquía conservada
`asic/strategy_B/config.yaml`	Memoria unificada FF, con reparación de antena
`asic/strategy_A_opt/config.yaml`	Variante `ESPINO_REG_DECODE`
`asic/p2_original/`	RTL del Proyecto 2 sin adaptar (evidencia de la sección 2.3)
`asic/pin_order.cfg`, `asic/pin_order.md`	Orden de terminales y su explicación
`asic/scripts/sweep.py`, `metrics.py`	Barridos y resumen de métricas con criterio de cierre
`asic/scripts/crit_path.py`	Ruta crítica con módulo y etapa
`asic/scripts/area_by_module.py`	Área por módulo con jerarquía completa
`asic/scripts/highlight_layout.py`	Layout anotado (banco, ALU, contador)
`sim/asic/tb_asic.v`, `sim/asic/stim.txt`, `asic/scripts/sim.sh`	Simulación RTL vs post-síntesis
`tools/rvasm.py`	Assembler: genera ROM y memoria, reporta uso

ANEXO B. EVIDENCIA ENTREGADA

Requisito	Dónde
☑️ Log y línea de cada incompatibilidad del RTL FPGA o explicación si no hubo mensaje	Sección 2.3, figuras 2 y 3
☑️ Área, FF, utilización y frecuencia máxima de A y B	Sección 3.3, figuras 5 y 6
☑️ Cinco celdas más frecuentes; áreas de banco, ALU y contador	Sección 4
☑️ Barrido de cinco períodos con setup y hold por corner	Sección 5.1
☑️ Ruta crítica real y medición antes/después de la optimización	Secciones 5.2 y 5.3
☑️ Cuatro utilizaciones y gráfico de congestión / violaciones	Sección 6
☑️ DRC, LVS, antena y captura de layout con instancias anotadas	Sección 7
☑️ RTL y post-síntesis con los mismos estímulos en ciclos	Sección 8
☑️ Versiones exactas del entorno y revisión del PDK	Sección 9 y README

ANEXO C. PROBLEMAS ENCONTRADOS Y CÓMO SE RESOLVIERON

Problema	Causa	Solución
PDK gf180mcuD was not found	La clave PDK del config no dispara la descarga	Pasar `--pdk gf180mcuD` en la línea de comandos (incluido en el Makefile)
Error en `pin_order.cfg`: *'#' requires a direction*	Los comentarios con # se leen como lado del die	Quitar comentarios; documentar en `pin_order.md`
Antena en B (red `_05410_`)	Pista larga de Metal2 hacia una compuerta	Reparación en ruteo global y detallado + diodos heurísticos (sección 7.2)

Problema	Causa	Solución
SIGKILL en el STA de B	WSL sin memoria (3,6 GB)	`.wslconfig`: 5 GB + 8 GB swap; `--jobs 2`
70 % de utilización no rutea	Faltan pistas de metal	Se elige 62 % como mínimo válido
`sim.sh`: *Permission denied*	Script sin permiso de ejecución	`chmod +x` (y `git update-index --chmod=+x`)